

Edge cases (context limit) — Hints

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Hint 3](#) · [Hint 4](#) · [Solution](#)

Work through in order — don't jump ahead until you've genuinely tried.

- [Hint 1](#)
- [Hint 2](#)
- [Hint 3](#)
- [Hint 4](#)

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Hint 3](#) · [Hint 4](#) · [Solution](#)

Hint 1

Simple Version

Every model has a maximum amount of text it can look at in one call — the "context window." If you send more than that, the API won't quietly trim your message for you — it will refuse the whole call and give you back an error.

Your job here: make a message so long it goes past that limit on purpose, so you see the real error with your own eyes instead of just reading about it.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Hint 3](#) · [Hint 4](#) · [Solution](#)

Intermediate Version

Each model has a fixed token budget for the whole request (system + user + history + the reply it's about to generate). Send a request whose token count exceeds that budget, and the API raises an error rather than silently truncating anything on your behalf — truncating your input for you would be a dangerous, silent behavior for an API to have, since it could quietly drop something important without telling you.

The easiest way to reliably blow past the limit is repetition: a single word repeated tens of thousands of times is cheap to generate in Python and produces a huge token count fast.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Hint 3](#) · [Hint 4](#) · [Solution](#)

Hint 2

Simple Version

- Build a giant string: something like one word repeated 200,000 times.
- Send it as the user message.
- Wrap the call in `try/except` so your program doesn't just crash — catch the error and print what kind it is.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Hint 3](#) · [Hint 4](#) · [Solution](#)

Intermediate Version

- `huge_input = "word " * 200_000` builds a string far larger than any current model's context window in a single line.
- Wrap the API call in `try: ... except openai.BadRequestError as e:` — a context-length problem surfaces as a `BadRequestError` (HTTP 400), not a special "context" exception of its own, so catching that specific type (not a bare `except:`) is what actually shows you which real exception class this is.
- Print `type(e)` and `str(e)` — the message text usually names the token counts involved directly, which is worth reading once for real.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Hint 3](#) · [Hint 4](#) · [Solution](#)

Hint 3

Simple Version

```
build a huge piece of text (a word repeated many, many times)
```

```
try:
```

```
    send it to the model
```

```
except (something went wrong):
```

```
    print what kind of error it was
```

```
    print the error's message
```

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Hint 3](#) · [Hint 4](#) · [Solution](#)

Intermediate Version

```
huge_input = "word " * 200_000

try:
    client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[{"role": "user", "content": huge_input}],
    )
except openai.BadRequestError as e:
    print(type(e).__name__)
    print(str(e))
```

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Hint 3](#) · [Hint 4](#) · [Solution](#)

Hint 4

Simple Version

```
huge_input = "word " * 200_000

try:
    client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[{"role": "user", "content": huge_input}],
    )
except Exception as e:
    print(type(e).__name__, str(e)[:200])
```

Run it — then try catching the *specific* error type instead of the bare `Exception` above, once you've seen what it actually is.

Intermediate Version

```
import openai

huge_input = "word " * 200_000

try:
    client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[{"role": "user", "content": huge_input}],
    )
except openai.BadRequestError as e:
    print(type(e).__name__, str(e)[:200])
```

Compare your output against the [Solution](#).

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Hint 3](#) · [Hint 4](#) · [Solution](#)

Full solution: [Show me the solution](#)